CY CERGY PARIS UNIVERSITÉ

UFR Sciences et Techniques

MASTER 1 INFORMATIQUE

Systèmes Intelligents et Communicants

Parcours Ingénierie des Données pour les
Systèmes Intelligents Distribués

INTELLIGENCE ARTIFICIELLE

TD 1 : WARGAME

Minimax et coupures Alpha-Bêta

Rapport de travaux dirigés

Réalisé par

Lisa OUYAHIA

Année universitaire 2026–2027

Octobre 2026

1. Conception et implémentation de l'IA

1.1 Fonction d'évaluation : f_eval()

Avant d'utiliser Minimax, j'ai dû donner à l'IA un moyen de comparer les positions. Ma fonction `f_eval()` commence par vérifier si la partie est terminée : elle renvoie `1000` en cas de victoire et `-1000` en cas de défaite. Sinon, elle combine la force des pions et leur progression sur le plateau.

```
int f_eval(Pion* jeu, int joueur)
{
    int g = f_gagnant(jeu);
    if(g == joueur) return 1000;
    if(g == -joueur) return -1000;

    return 10*(f_valeur(jeu,joueur)
              - f_valeur(jeu,-joueur))
         + f_avance(jeu,joueur)
         - f_avance(jeu,-joueur);
}
```

J'ai choisi de multiplier la différence des valeurs des pions par `10`, afin que perdre une unité coûte davantage qu'un simple déplacement.

J'ai aussi ajouté `f_avance()` : pour les pions `x`, elle compte les lignes parcourues vers la ligne 0 ; pour les pions `o`, elle compte la progression vers la ligne 9.

Sans ce critère, plusieurs coups sans bataille obtenaient le même score et l'IA conservait simplement le premier trouvé.

Les coefficients ont été choisis à la main, sans optimisation automatique.

1.2 Fonctions f_min() et f_max()

J'ai complété les deux fonctions récursives de Minimax.

`f_max()` recherche le meilleur score pour le joueur de l'IA et `f_min()` simule les réponses de l'adversaire en retenant le score le moins favorable à l'IA.

Pour chaque pion du joueur concerné, je teste les cases voisines. Je copie d'abord le plateau avec `f_copie_plateau()`, puis j'utilise `f_bouge_piece()` sur cette copie.

Si le déplacement est valide, les fonctions s'appellent à tour de rôle, avec une profondeur diminuée de 1.

La recherche s'arrête à la profondeur 0 ou dès qu'une victoire est détectée, et la position est alors évaluée avec `f_eval()`.

J'ai fixé `PROFONDEUR` à `3`. L'IA anticipe donc son coup, la réponse adverse et sa propre réponse suivante.

Le paramètre `joueur` reste inchangé d'un appel à l'autre pour que tous les scores soient calculés du même point de vue.

1.3 Choix d'un coup : f_IA()

La fonction `f_IA()` parcourt tous les déplacements autorisés, sans modifier immédiatement le vrai plateau.

Après avoir simulé chaque coup, elle appelle `f_min()` avec une profondeur de `PROFONDEUR-1`, puisque le premier demi-coup vient d'être joué.

```
valeur = f_min(copie, joueur,
               PROFONDEUR-1,
               meilleur, INFINI);

if(valeur > meilleur) {
    meilleur = valeur;
    bl = l; bc = c;
    bdl = dl; bdc = dc;
}
```

Je mémorise les coordonnées du meilleur déplacement, puis je le joue une seule fois sur le plateau réel.

Si deux coups ont le même score, je garde le premier rencontré, car la comparaison utilise `>` et non `>=`.

2. Optimisation avec Alpha-Bêta

Minimax examine beaucoup de positions. J'ai ajouté les coupures Alpha-Bêta pour arrêter l'exploration d'une branche dès qu'elle ne peut plus changer le choix final.

`alpha` est la meilleure borne connue pour MAX ; `beta` est celle de MIN.

Dans `f_max()`, je mets à jour `alpha` lorsque le meilleur score augmente.

Dans `f_min()`, je mets à jour `beta` lorsque le score diminue.

Dès que `alpha >= beta`, la branche peut être coupée.

```
if(meilleur > alpha)
    alpha = meilleur;

if(ALPHA_BETA && alpha >= beta) {
    free(copie);
    return meilleur;
}
```

J'ai utilisé la constante `ALPHA_BETA` : lorsqu'elle vaut `1`, les coupures sont actives ; lorsqu'elle vaut `0`, elles sont désactivées.

Cela permet de garder la même recherche et la même fonction d'évaluation, tout en changeant seulement l'optimisation.

3. Expérimentations et analyse

3.1 Tests et mesures

Pour mesurer le travail effectué par l'IA, j'ai ajouté un compteur `nbNoeuds`, incrémenté dans `f_min()` et `f_max()`.

Je mesure aussi le temps CPU avec `clock()` et j'affiche les deux valeurs après la recherche.

J'ai testé le mode humain contre IA. Dans un de mes essais, le joueur humain joue `F9F8`, puis l'IA répond `A0A1`.

Mes journaux contiennent notamment les mesures suivantes, issues de différentes étapes du développement :

Essai	Nœuds visités	Temps CPU
1	87 567	0,171 s
2	2 075	0,004 s
3	2 099	0,005 s

Entre les deux premières mesures, l'écart est de **85 492 nœuds**, soit environ **97,63 % de nœuds en moins**.

Le temps affiché passe de `0,171 s` à `0,004 s`.

C'est un écart important, mais ces exécutions ont été enregistrées à différents moments : je ne les présente donc pas comme une comparaison rigoureuse de la version finale avec et sans Alpha-Bêta.

Pour cela, il faudrait rejouer exactement la même position avec uniquement `ALPHA_BETA` modifié.

3.2 Vérifications fonctionnelles

J'ai également vérifié la saisie des déplacements et le contrôle des coups interdits : lorsqu'un mouvement n'est pas autorisé, le programme affiche `mauvais choix`.

J'ai testé le mode humain contre humain jusqu'à la fin d'une partie ; le message final était `joueur x gagne!`.

Ces essais m'ont permis de vérifier le déroulement du jeu et la détection du gagnant.

Le programme propose également un mode IA contre IA, mais je n'ai pas de mesures complètes et comparables sur ce mode dans les résultats présentés ici.

4. Difficultés, limites et conclusion

J'ai d'abord corrigé des problèmes dans le programme de départ, notamment la variable `retour` dans `f_test_mouvement()` et une erreur liée à `f_humain()`.

La difficulté principale a ensuite été de simuler les coups sans modifier le vrai plateau.

Pour cela, j'ai travaillé avec des copies et fait en sorte que les fonctions de bataille et de victoire utilisent le plateau reçu en paramètre.

Mon IA reste limitée à trois demi-coups et à une heuristique simple. Elle ne voit pas les menaces lointaines.

L'ordre fixe des déplacements limite aussi l'efficacité possible des coupures.

Pour aller plus loin, je pourrais examiner en priorité les coups prometteurs, augmenter la profondeur et enrichir la fonction d'évaluation.

Au final, ce TD m'a permis de mettre en œuvre Minimax, de comprendre l'intérêt des coupures Alpha-Bêta et de mesurer le coût de la recherche.

Les tests ont surtout montré l'importance d'une bonne fonction d'évaluation et d'une comparaison expérimentale réalisée dans les mêmes conditions.

Dépôt GitHub :

https://github.com/lisaouy/TD1-IA-Wargame